表 7.4: USB 键盘真机排错的九个阶段

	现象	根因	修复
1	port 3 状态 `0x301`，复位后 `0x303`	低速设备在高速 Hub 后面	记录父 Hub、端口、速度，走 split 事务
2	`control setup failed req=6 addr=0, intr=0x0a`	SETUP 的 SSPLIT 没设 `LSPDDEV`，TT 回 STALL	按 `usb_route` 给所有低速事务置 `LSPDDEV`
3	GET_DESCRIPTOR 数据阶段返回 −2（超时）	CSPLIT 收到 NAK 后仍反复发 CSPLIT	NAK 时清 `complete_split`，下一帧重发 SSPLIT；NYET 才继续 CSPLIT
4	设备描述符 `class=0 vid=0 pid=0`	18 字节跨 3 个 8 字节包，被当成一个 split 请求，只收到前 8 字节	按最大包长分包，每包单独 SSPLIT/CSPLIT，完整收到一包后翻转 DATA0/1；检查长度、类型和 VID
5	“not a boot keyboard”	复合设备的后一个接口覆盖了键盘接口的属性	先定位 Boot Keyboard 接口，端点只在该接口内选
6	ready 之后又出现 `req=6 addr=4` STALL	绑定键盘后仍继续 CDC-ECM 网卡探测，向键盘要配置描述符	HID 交给 USB 总线后直接返回
7	按键无响应，反复 `hcint=0x42`	把每个 NYET 都当作“TT 还在忙”，永久重发 CSPLIT	记录 SSPLIT ACK 的帧号，同帧内最多重试 3 次；跨帧则结束本轮，等 `bInterval`
8	`ssack` 比 `issue` 晚几十个微帧	ACK 交给 kworker 处理，调度延迟毫秒级，CSPLIT 窗口早过了	SSPLIT ACK 和窗口内 NYET 改在 IRQ 快速路径里处理
9	快速路径启用后仍偶尔跨帧	快速路径之前有一行 `printf`；115200 波特率下一行要几毫秒	中间状态不打印，只在本轮结束后限量打印 `ch6 final`

阶段 7 ~ 8：时间窗口。 周期性的 split 事务有严格的时序：CSPLIT 应在 SSPLIT 之后第 2 个微帧开始，并且只能在同一个帧内重试。早期代码对 NYET 无条件重试，永远等不到结果；改成按帧号判断窗口后，又发现记录下来的“ACK 帧号”比实际晚了几十个微帧——因为 ACK 是由 kworker 处理的，而 kworker 要等调度器给它 CPU，延迟是毫秒级的，比 125 μs 的微帧粗了一个数量级。解决办法是把这两种有截止时间的中间事件移到中断上下文里（`dwc2_interrupt_rx_irq_fast()`），只把“这一轮已经结束”的结果交给线程。这是“上半部只做最少的事”这条原则的一个反例：对有硬实时要求的状态推进，上半部恰恰是唯一来得及的地方。

阶段 9：观察者效应。 快速路径加上以后，问题偶尔还在。原因是 `dwc2_irq()` 开头还留着一行调试 `printf`：串口 115200 波特率下每个字符约 87 μs，一行几十个字符就是好几个毫秒，足够错过整个窗口。这是典型的“加日志就改变时序”的 Heisenbug。最终版本在有截止时间的分支里一行都不打印，只在本轮结束之后限量打印 `dwc2 ch6 final`；如果确实需要跟踪中间事件，应该写进内存里的 trace 环形缓冲，事后由进程上下文打印。

限量日志各自证明什么。 排错时最容易把“请求已经发出”当成“数据已经回来”，所以这四行日志的边界要背下来：

- `dwc2: ch6 armed addr=4 ep=1 split=...`：HCSPLT、HCCHAR、HCTSIZ、HCINTMSK(6)、HAINTMSK 和 GINTMSK 都已配置，channel 已使能。仅此而已，键盘的一个字节都还没到。
- `dwc2 ch6 final p=... int=... uf=... issue=... age=... ssack=...`：本轮已经走到“不再需要满足当前 CSPLIT 实时期限”的出口才会打印。
- `usbkbd: channel 6 IRQ received`：HCD 已归还 URB，键盘的 completion 把工作排进了 `usbkbd_wq`。
- `usbkbd: first HID report mod=0 keys=10,0,0,0,0,0`：workqueue 读到了 DMA 缓冲里的报告；`keys=10` 是十六进制的 usage `0x10`，也就是字母 `m`，`mod=0` 表示没有修饰键。

`ch6 final` 那一行的字段就是为阶段 8 和 9 准备的：`p` 是本轮处在哪个 split 阶段（`p=1` 表示 complete-split），`int` 是 `HCINT`，`uf` 是 IRQ 采到的微帧号（`HFNUM` 低 3 位），`issue` 是本次 channel 使能时的计数器，`age` 是从发出到 IRQ 经过的微帧数，`ssack` 是最近一次 SSPLIT 被 ACK 时的计数器。对比 `issue`、`age` 和 `ssack` 就能分辨“总线慢”还是“软件来晚了”。因为中间状态一律不打印，这行日志还做了限量（前 12 条），空闲键盘的 NAK 轮次也不消耗这个额度。`HCINT` 常用取值的解码见表 7.3：`0x22` 是 `CHHLTD|ACK`（SSPLIT 已被 TT 接受），`0x42` 是 `CHHLTD|NYET`（CSPLIT 还没有结果），`0x03` 才是 `CHHLTD|XFERCOMPL`（DMA 里是完整报告）。

诊断顺序。 整理下来，按键没反应时按这个顺序看日志：有没有 `ch6 armed`（channel 配置并启动了）；按键时有没有 `ch6 final`（host channel 产生了中断）；有没有 `usbkbd: channel 6 IRQ received`（URB 已归还、键盘 work